

一、为什么我们需要LlamaIndex?

--楼兰

章节目标：认识LlamaIndex框架。初步理解LlamaIndex的核心价值。

LlamaIndex是一个开源的Python库，用于构建和部署可扩展的AI应用程序。也就是说LlamaIndex是一个帮助程序员构建AI应用的框架。这样的框架其实有很多，有通用的LangChain，也有专门针对某一些特定领域的框架，比如阿里云百炼的Dashscope等。LlamaIndex和这些框架一样，都是帮助我们构建基于AI的应用的。

在接触LlamaIndex之前，我们有必要先了解一下什么是AI应用，以及这些框架要帮我们做什么事情，为什么我们会需要这些大模型框架。

一、与大模型交互的方式

谈到这些大模型框架的作用，很多朋友的第一印象是，这些框架可以帮助我们和大语言模型进行交互。但是，实际上，所有主流大语言模型，都是通过HTTP协议和客户端进行交互。

例如，下面的案例，只需要一个curl工具就可以访问阿里云百炼上的通义千问大模型：

```
!curl -X POST https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions \-H
"Authorization: Bearer sk-a6659clef6a640e7961426a8bd80175c" \-H "Content-Type:
application/json" \-d '{"model": "qwen-plus", "messages": [{"role": "system", "content":
"You are a helpful assistant."}, {"role": "user", "content": "怎么退款? "} ]}'
```

这里使用的curl工具就是使用我们常见的HTTP协议和通义千问进行交互的。我们并不需要那些复杂的框架，只要一个支持HTTP协议的工具，就可以完成和大语言模型的交互了。

例如，下面就是一个使用request库，以HTTP协议访问大语言模型的案例。

```
import requests
import json

url = "https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions"
headers = {
    "Authorization": "Bearer sk-a6659clef6a640e7961426a8bd80175c",
    "Content-Type": "application/json"
}
data = {
    "model": "qwen-plus",
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "怎么退款? "}
    ]
}

response = requests.post(url, headers=headers, json=data)

try:
    response.raise_for_status() # 检查请求是否成功
```

```
print(json.dumps(response.json(), indent=2, ensure_ascii=False))
except requests.exceptions.RequestException as e:
    print(f"请求出错: {e}")
    if response.text:
        print(f"错误详情: {response.text}")
```

二、构建Agent，定制大模型的业务能力

从上面可以看到，大语言模型的客户端框架并不是帮我们实现和AI的交互，而是帮我们更轻松的构建AI应用。其实核心就是两件事：

- 简化和大语言模型交互的过程。
- 利用大语言模型快速处理复杂的问题。

而LlamaIndex也是这样一个框架。

```
# 安装LlamaIndex依赖
! pip install llama-index
# 安装LlamaIndex集成Langchain的依赖
! pip install langchain
! pip install langchain_community
! pip install dashscope
! pip install llama-index-llms-langchain
! pip install llama-index-embeddings-langchain
! pip install llama-index-llms-dashscope
! pip install llama-index-embeddings-dashscope
```

```
from llama_index.core.agent.workflow import ReActAgent
from llama_index.core.workflow import Context
from llama_index.llms.langchain import LangChainLLM
from langchain_community.chat_models import ChatTongyi
from config.load_key import load_key
from pydantic.types import SecretStr

llm = LangChainLLM(ChatTongyi(
    model="qwen-plus",
    api_key=SecretStr(load_key("BAILIAN_API_KEY")),
))
agent = ReActAgent(llm=llm)
ctx=Context(agent)
```

```
resp = await agent.run("怎么退款?", ctx=ctx)
resp
```

通过这样的封装，应用中就只需要关注Agent，并以此来构建业务场景。而不需要太多的关注内部交互的细节。

当然，通常构建Agent时，不只是简单的与大语言模型进行交互。还需要增加Memory和Tools。

Memory和Tools都是大语言模型原生支持的功能。其中Memory是给大语言模型提供历史上下文记忆功能，这样我们在询问大语言模型时，可以让大语言模型理解历史的聊天信息，更好的理解问题。Tools则是给大语言模型提供本地工具。这样，大语言模型就可以通过工具来完成一些具体的任务。

```
from llama_index.core.memory.chat_memory_buffer import ChatMemoryBuffer

def get_weather(city: str) -> str:
    """获取某个城市的天气"""
    print(f">>>> 正在获取{city}的天气...")
    return f"城市: {city}, 天气一直都是晴天!"

agent2 = ReActAgent(llm=llm, tools=[get_weather])
ctx2=Context(agent2)
memory = ChatMemoryBuffer.from_defaults(token_limit=4000)
```

```
resp = await agent2.run("长沙天气怎么样?", ctx=ctx2, memory=memory)
print(resp.response)
resp = await agent2.run("北京呢?", ctx=ctx2, memory=memory)
print(resp.response)
```

实际上，大语言模型本身所有的功能都是通过HTTP协议对外公布的。具体可以参考阿里云百炼上的说明：https://bailian.console.aliyun.com/console?tab=api#/api/?type=model&url=https%3A%2F%2Fhelp.aliyun.com%2Fdocument_detail%2F2712576.html

而包括LlamaIndex、LangChain甚至包括Java语言当中的SpringAI等客户端框架，最终都是通过HTTP协议和模型进行交互的。

三、使用LlamaIndex快速实现RAG应用

这种通过Agent智能体构建大模型应用的方式，在很多业务场景中，都是非常有效的。不过，Agent却并不是LlamaIndex的重点。一方面，现在业界有LangChain框架，更适合构建Agent智能体。另一方面，除了Agent，业界还有大语言模型更重要的使用场景RAG。

RAG全称Retrieval Augmented Generation，检索增强生成。这是一种大模型应用落地最成熟的方式。RAG的核心思想是在询问大语言模型问题时，给大语言模型提供更多本地信息作为参考，从而让大语言模型能够更好的理解用户的问题，给出更有针对性的答案。

回到之前的案例，我们希望大语言模型回答“怎么退款”这个问题时，其实最需要的，还不是提供Agent，而是希望给大语言模型提供我们自己业务平台的业务规则，这样大语言模型才能根据业务规则，给出更符合业务场景的答案。

RAG的具体实现流程，我们会在后面章节详细分析，这里，我们先给出一个LlamaIndex的案例，你可以先体会一下使用LlamaIndex实现RAG是一件多么简单的事情。

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from langchain_community.chat_models import ChatTongyi
from config.load_key import load_key
from llama_index.embeddings.dashscope import (
    DashScopeEmbedding,
    DashScopeTextEmbeddingModels,
    DashScopeTextEmbeddingType,
```

```

)

# 初始化通义千问的Embedding模型
Settings.embed_model=DashScopeEmbedding(
    model_name=DashScopeTextEmbeddingModels.TEXT_EMBEDDING_V2,
    text_type=DashScopeTextEmbeddingType.TEXT_TYPE_DOCUMENT,
    api_key=load_key("BAILIAN_API_KEY"),
)

Settings.llm=ChatTongyi(
    model="qwen-plus",
    api_key=SecretStr(load_key("BAILIAN_API_KEY")),
)

#加载参考数据
documents = SimpleDirectoryReader("resource").load_data()
index = VectorStoreIndex.from_documents(documents)

# 构建检索引擎
query_engine = index.as_query_engine()
response = query_engine.query("怎么退款")
response

```

在这个过程中，我们实际上是先完成了对本地文件的检索，找到了和问题相关的一些知识条目。然后，将这些知识条目和用户的问题一起发给了大语言模型，这样大语言模型就能够更精准的回答问题。

四、总结

首先，通过这一章节的演练，可以看到，LlamaIndex和LangChain这些大语言模型框架，最主要的价值有两个

- 简化和大语言模型交互的实现过程
- 结合大语言模型，实现更复杂的业务场景

所以，对这些大语言模型框架，其实不应该分开来理解，而应该将它们看做是一个整体。

然后，我们用LlamaIndex演示了一个Agent的案例，通过Agent能够更好的扩展大语言模型的能力边界。

接下来，我们又介绍了另外一种扩展大语言模型边界的方式：RAG。而这正是LlamaIndex最大的强项，也是后面分享的重点。

最后，其实LlamaIndex也并不是一个独立的框架。实际上，在最后的案例中，我们使用LlamaIndex针对阿里云Dashscope的扩展，实现了RAG功能，但是，其实通过LlamaIndex的良好设计，也可以很轻松的集成LangChain框架，然后通过LangChain强大的功能扩展，快速接入更多其他的主流大语言模型。